

Final Term Project

OBJECT-ORIENTED PROGRAMMING

Student names and ID:-

- ❖ Hasan Ali (F25BDATS1M02078)
- ❖ Fatima Manzoor (F25BDATS1M02087)
- ❖ Amna Ejaz (F25BDATS1M02085)

Chosen Library :-

- ❖ The Matplotlib Library

Submission Date :-

- ❖ May 28, 2026

OOP Analysis of the matplotlib Library

Section 1 — Library Overview

Matplotlib is Python's most popular and widely used Library. It is used for 2D visualization of objects. It was created in 2003 by John D. Hunter. The library is used in many fields like scientific research, machine learning, financial analysis, engineering, and academia etc. Major companies like NASA rely on matplotlib for making their graphs and charts.

What matplotlib Produces

- Line charts
- Bar charts
- Scatter plots
- Histograms
- Pie charts
- Heatmaps

Installation

To install we need to run this command in terminal :-

```
pip install matplotlib
```

Two Interfaces

Matplotlib provides two distinct ways to create charts.

pyplot interface

- For Quick plots, beginners, scripts

OOP interface

- For Professional use, this assignment, complex charts

Theoretical Deep-Dive into the Dual-Interface Paradigm

The architectural layout of Matplotlib provides two completely separate control pathways for data visualization, designed around different programming paradigms:

1. 1. Procedural Interface (pyplot):

This interface works like MATLAB. When you use commands such as `plt.plot()`, Matplotlib automatically creates and manages figures in the background. It is simple and useful for quick plotting, but it relies on global states, which can make large or complex projects harder to manage.

2. The Explicit Object-Oriented Interface:

This interface exposes Matplotlib's underlying class architecture directly to the developer. Rather than relying on background state trackers, it requires explicit instantiation of structural object containers through structural factory design methods.

```
fig, ax = plt.subplots()
```

Here, `fig` is a distinct instance of the `matplotlib.figure.Figure` class, and `ax` is an instance of the `matplotlib.axes.Axes` class. All visualization updates must be explicitly invoked as instance method calls targeting these specific variables. This design provides maximum modularity, allows components to be completely isolated within independent software scopes, and makes it simple to embed visualization structures directly into standard enterprise GUI frameworks (such as PyQt, Tkinter, or wxPython).

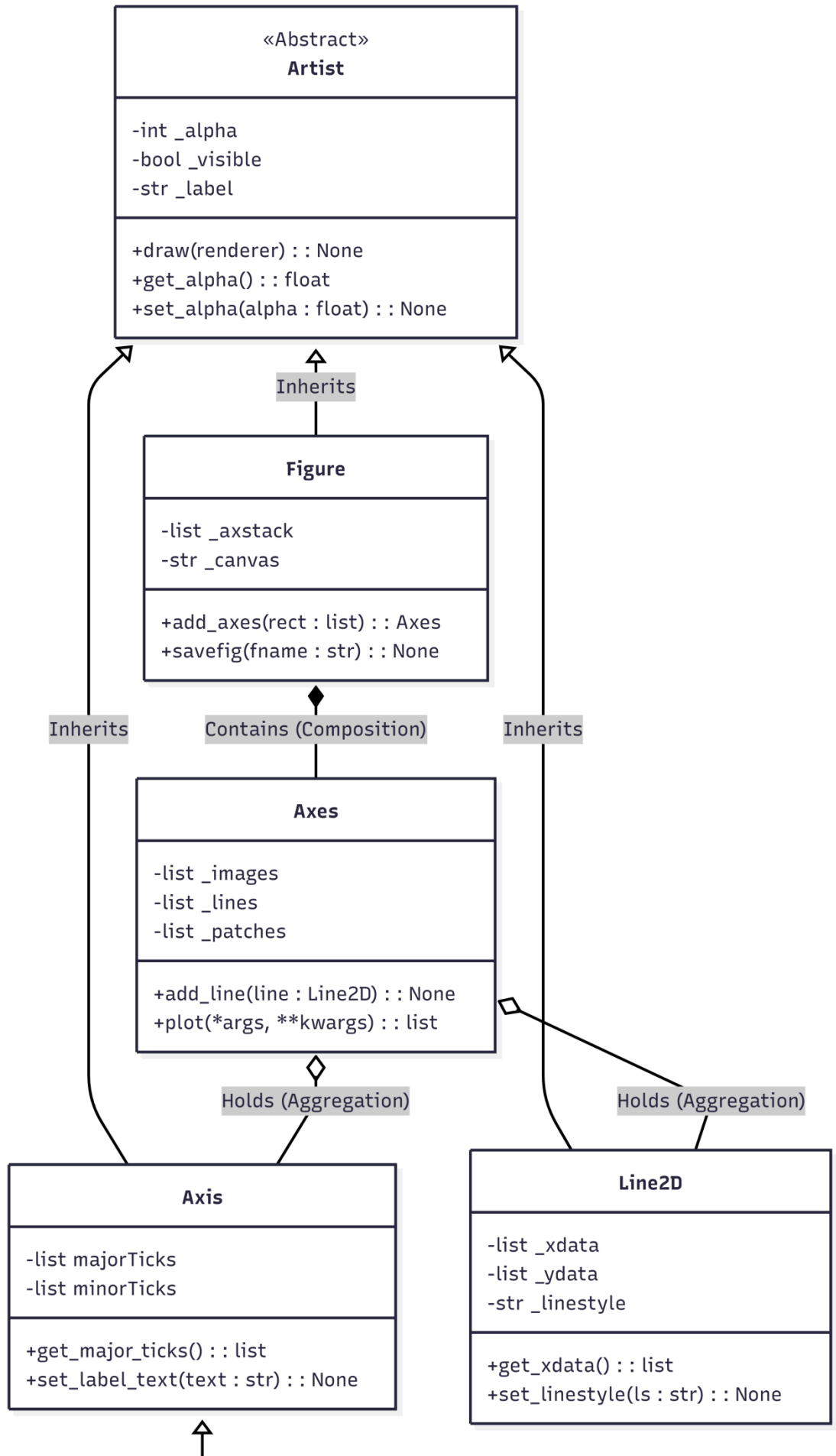
Section 2 — Class Hierarchy Diagram

Matplotlib library is built on a single base class called Artist. Every visible element in a matplotlib figure, the window, the chart area, lines, shapes, labels, tick mark is a subclass of Artist. All other classes inherits from Artist class in the whole library.

This is a UML diagram represent relation between some classes of matplotlib.

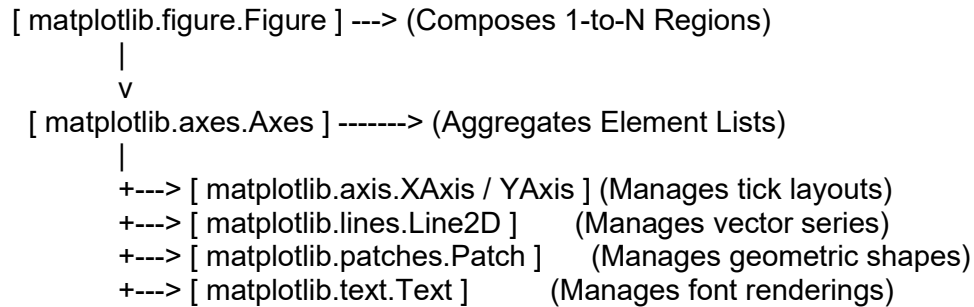
Key Class Descriptions

Artist: Abstract base for all visual elements
Figure: Top-level window/canvas container
Patch: Base for Rectangle, Circle, FancyArrow
Axes: One chart area inside a Figure



Structural Composition and Aggregation Analysis

The visual class hierarchy of Matplotlib operates as an explicit tree structure managed by composition and aggregation patterns. Every component is structurally mapped beneath the top-level canvas management layer:



- **Figure Class:** The Figure acts as the main window or canvas that contains all visualizations. It manages the overall plot structure. If a Figure object is deleted, all plots and elements inside it are removed as well.
- **Axes Class:** The Axes represents the actual plotting area where data is displayed. It manages chart elements such as lines, shapes, labels, and titles, making it the main component responsible for drawing and organizing the visualization.

Section 3 — OOP Principles Analysis

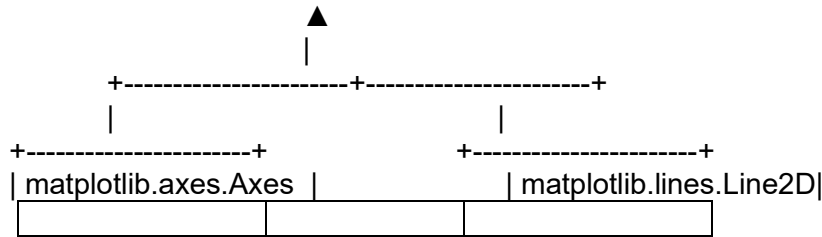
Now we gonna Find OOP principles from different source codes of different classes from matplotlib library. We are going to cover all 4 of OOP principles.

1 Inheritance:

Inheritance is the most prominent OOP feature in matplotlib. Every visible element in the library Figure, Axes, Line2D, Text, Rectangle, Circle inherits from the Artist base class. Here are some example of it 3.1 Inheritance Architecture

Inheritance within Matplotlib is structured around a single, unified abstract root class: `matplotlib.artist.Artist`. Every single visible object—from the high-level outer window box down to a single tick mark line or text label—is a direct or indirect subclass of this base class.





This structural generalization allows subclasses to instantly inherit a large set of standardized properties and configuration handles without code duplication. Every child component automatically acquires attributes such as tracking layers (`_zorder`), alpha transparency scales (`_alpha`), visibility states (`_visible`), and transformation matrix paths. This design ensures that all canvas objects interact with the global rendering engine through a completely uniform interface.

```

1  from collections import namedtuple
2  import contextlib
3  from functools import cache, reduce, wraps
4  import inspect
5  from inspect import Signature, Parameter
6  import logging
7  from numbers import Number, Real
8  import operator
9  import re
10 import warnings
11
12 import numpy as np
13
14 import matplotlib as mpl
15 from . import _api, cbook
16 from .path import Path
17 from .transforms import (BboxBase, Bbox, IdentityTransform, Transform, TransformedBbox,
18                          TransformedPatchPath, TransformedPath)
19
20 _log = logging.getLogger(__name__)

```

In this code we can see this class is inheriting many methods or many other classes by using import. Inheritance is most commonly used in matplotlib library to get data of one class in another class.

2 Encapsulation:

Encapsulation is also a important and widely used principle of OOP. Encapsulation means hiding or protecting anything. It is basically used to hide complex backend data from users or for protecting data inside a class. In matplotlib's Figure class, the internal list of axes is stored as a

private attribute.

```
121     artists to the figure or subfigure, create Axes, etc.
122     """
123     def __init__(self, **kwargs):
124         super().__init__()
125         # remove the non-figure artist _axes property
126         # as it makes no sense for a figure to be _in_ an Axes
127         # this is used by the property methods in the artist base class
128         # which are over-ridden in this class
129         del self._axes
130
131         self._suptitle = None
132         self._supxlabel = None
133         self._supylabel = None
134
135         # groupers to keep track of x, y labels and title we want to align.
136         # see self.align_xlabels, self.align_ylabels,
137         # self.align_titles, and axis._get_tick_boxes_siblings
138         self._align_label_groups = {
139             "x": cbook.Grouper(),
140             "y": cbook.Grouper(),
141             "title": cbook.Grouper()
142         }
143
144         self._localaxes = [] # track all Axes
145         self.artists = []
146         self.lines = []
147         self.patches = []
148         self.texts = []
149         self.images = []
```

In the given image we can see use of encapsulation by identifying protected methods using underscore.

Encapsulation and State Protection:

Matplotlib follows the principle of **encapsulation** to keep its internal data secure and maintain stability. Important information is stored in protected variables (such as ``self._xaxis`` and ``self._label``) that should not be accessed directly.

Instead, users interact with objects through public methods, which safely update their properties:

```
ax.set_title("System Diagnostic Report")
```

```
ax.xaxis.set_label_text("Time Interval (Seconds)")
```

This approach helps prevent errors and keeps the visualization components properly managed.

3 Polymorphism:

Polymorphism means the same method name behaves differently depending on which object it is called on. In matplotlib, every single Artist subclass has a draw method. The rendering engine simply calls draw() on each object, it does not need to know or check whether it is a Figure, Axes, Line2D, or any other type. Each one draws itself differently. This thing uses polymorphism means many shapes of a thing.

```
153         self.suppressComposite = None
154         self.set(**kwargs)
155
156     def _get_draw_artists(self, renderer):
157         """Also runs apply_aspect"""
158         artists = self.get_children()
159
160
396         'ha': 'center', 'va': 'top', 'rotation': 0,
397         'size': 'figure.titlesize', 'weight': 'figure.titleweight'}
398     return self._suplabels(t, info, **kwargs)
399
400     def get_suptitle(self):
401         """Return the suptitle as string or an empty string if not set."""
402         text_obj = self._suptitle
403         return "" if text_obj is None else text_obj.get_text()
404
405     @_docstring.Substitution(x0=0.5, y0=0.01, name='super xlabel', ha='center',
406                             va='bottom', rc='label')
407     @_docstring.copy(_suplabels)
```

Here we can see some use of polymorphism in matplotlib.

Runtime Polymorphism and Dynamic Rendering Dispatch:

Polymorphism helps Matplotlib draw different chart elements efficiently. The canvas does not need to check the type of every object separately. Instead, it calls the same .draw() method for all objects:

```
for child in self.get_children():
    child.draw(renderer)
```

The actual action depends on the object's type:

- **Line2D** objects draw lines and curves.
- **Patch** objects draw shapes such as rectangles and polygons.
- **Text** objects render labels, titles, and other text.

This allows different objects to be handled through a common interface while performing their own specific drawing tasks.

4 Abstraction:

Abstraction means hiding complex internal implementation behind a simple interface. When you call `ax.plot()`, you are asking for a line to be drawn. Behind the scenes, matplotlib creates a `Line2D` object, converts your data coordinates to pixel coordinates, registers it with the axes, manages z-ordering, handles color cycles, and dozens of other operations. None of this complexity is exposed to the user.

```
112     def __setstate__(self, state):
113         next_counter = state.pop('_counter')
114         vars(self).update(state)
115         self._counter = itertools.count(next_counter)
116
117
118  ✓ class FigureBase(Artist):
119     """
120     Base class for `.Figure` and `.SubFigure` containing the methods that add
121     artists to the figure or subfigure, create Axes, etc.
122     """
123  ✓ def __init__(self, **kwargs):
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503     def add_artist(self, artist, clip=False):
504
505
506
507
508
509
510
511
512
513
514
515
516
517         clip : bool, default: False
518             Whether the added artist should be clipped by the figure patch
519
520         Returns
521         -----
522         ~matplotlib.artist.Artist
523             The added artist.
524     """
525     artist.set_figure(self)
526     self.artists.append(artist)
527     artist._remove_method = self.artists.remove
528
```

Deep Abstraction and Structural Complexity Masking:

Abstraction separates high-level user goals from complex low-level code operations. This is clearly demonstrated by the standard plotting method:

```
ax.plot([1, 2, 3], [4, 5, 6], color='red', linestyle='--')
```

This single method call completely abstracts a massive sequence of internal operations:

1. Validating the dimension boundaries and data types of the input arrays.
2. Generating a new, separate instance of the `matplotlib.lines.Line2D` class.
3. Automatically injecting the line instance into the `Axes.lines` private registry array.
4. Parsing text parameters (' -- ') into low-level dash patterns.
5. Calculating pixel scale transformations to map data units onto the screen.
6. Triggering standard container update workflows across the display system.

Section 4 — Design Decision Critique

The most noticeable design in matplotlib library is every single class inherits from a single class Artist. Because of this, Figure, Axes, Line, Text, Rectangle, and all other visual objects follow the same system. The Artist class provides common features such as `draw()`, `alpha`, `visible`, and `zorder`.

This design has several advantages. The rendering system can handle every object in the same way by simply calling the `draw()` method without checking the object type. It also provides a consistent API because the same methods like `set_visible()` and `set_alpha()` work on all visual elements. In addition, the system is extensible because developers can create new visual elements by inheriting from Artist.

The only bad thing here is because Artist class holds all data it become a large class. This violates the Single Responsibility Principle, which states that a class should have only one responsibility. The deep inheritance hierarchy also makes the code difficult to understand because methods may come from multiple parent classes. Also the entire library depends heavily on Artist so any change in the Artist class can affect almost every other class in matplotlib.

A more modern alternative would be to use a Protocol instead of inheritance. In this approach, any class that implements a `draw()` method would automatically work with the rendering system without needing to inherit from Artist.

Section 5 — Comparison with plotly

To appreciate matplotlib's OOP design, it is valuable to compare it with plotly an alternative Python visualization library that solves the same problem but with a fundamentally different design approach.

OOP Design Comparison

OOP Hierarchy:

Matplotlib:	Artist -> Figure -> Axes -> Line2D etc. 10+ classes
Plotly:	primarily dict-based data structure, less class inheritance

Extensibility:

Matplotlib:	inherit from any class and add custom behavior
Plotly:	extending chart objects is difficult

Interactivity:

Matplotlib: Static by default, requires extra libraries for interactivity
Plotly: Interactive out of the box hover, zoom, click built in

OOP for Study:

Matplotlib: all 4 principles clearly visible and demonstrable
Plotly: hard to find inheritance chains

Ecosystem:

Matplotlib: Seaborn, Pandas, scikit-learn all built on top of it
Plotly: Dash, Streamlit use plotly for web dashboards

5.2 Micro-Architectural Blueprint Evaluation

Feature Comparison Criteria	Matplotlib Library Design	Plotly Visualization Engine
Foundational Object Model	Explicit, stateful class tree rooted at the abstract <code>Artist</code> base.	Nested, declarative JSON-like dictionary configuration trees.
Component Extensibility	High; natively handles structural inheritance and method subclass overrides.	Low; requires rewriting serialization properties and JavaScript modules.
Runtime State Mechanics	Retains active, living object representations in memory for every component.	Re-initializes clean data frames dynamically upon event updates.
Primary Production Use Case	Highly complex, print-ready, static layouts or custom GUI integrations.	Web-native, interactive application dashboards and real-time feeds.

5.3 Comparative Architectural Essay

The fundamental split between Matplotlib and Plotly represents a classic software design trade-off between *stateful object-oriented hierarchies* and *declarative serialization structures*.

Matplotlib treats every single component on a chart as a living, stateful Python object instance. This requires a higher initial learning curve but provides unmatched structural control. Because every line, label, and axis line is a fully realized instance of a class, software engineers can use standard object-oriented design patterns to modify, intercept, or extend any part of the visualization engine. This makes Matplotlib an exceptional choice for academic study, research, and applications requiring precise, pixel-perfect customizations.

Conversely, Plotly relies on a declarative serialization model. A Plotly chart is essentially a deeply nested JSON dictionary structure that maps out configuration options as pure text strings and key-value fields. This data structure is then sent over a network boundary to be parsed and rendered on a webpage using an independent JavaScript engine (`Plotly.js`). While this model delivers excellent out-of-the-box web interactivity (such as panning, zooming, and hover tooltips), it restricts your ability to customize components using Python. You cannot easily subclass a Plotly element to change its internal behavior; instead, you are strictly limited to the pre-configured parameters exposed within its JSON specification.

Conclusion

For learning and showing OOP principles matplotlib is the better choice because its class structure clearly Express all major OOP concepts. Plotly mainly uses a dictionary based style so it is less useful for OOP study.

In data science both libraries are useful matplotlib is best for detailed static graphs while plotly is best for interactive charts and dashboards.

Section 6 — Custom Extension Code

In this section we are going to add custom extension in any class of matplotlib library, we are going to add some new methods and classes in Axis class. Firstly we wrote my-extension file where we wrote code of our extension which we do in Axis class. We basically added one new class and three new methods in it.

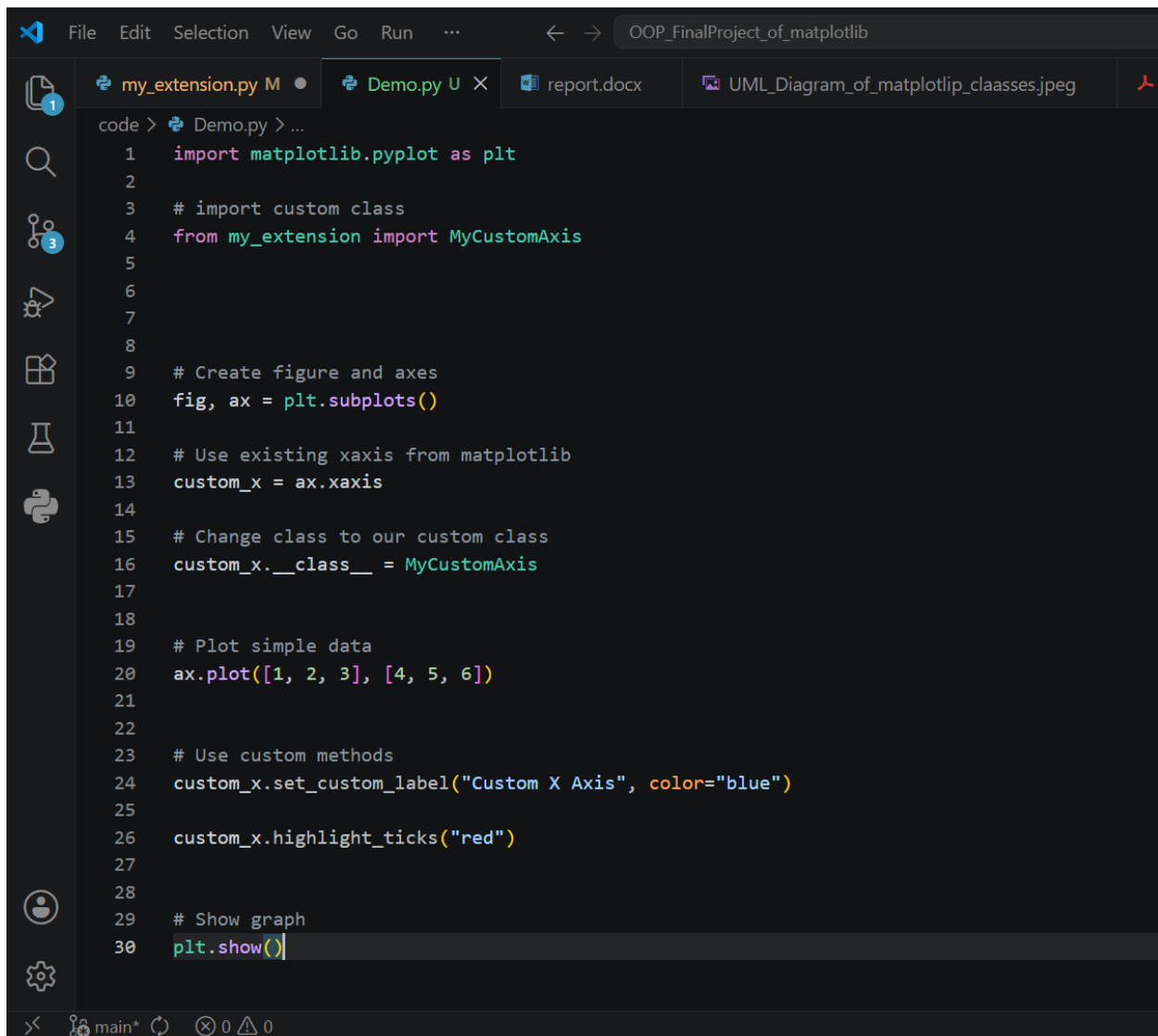
The class we added is “MyCustomAxis”

And methods are:

Method	Kya karta hai
<code>set_custom_label(text, color)</code>	To set color lable at Axis
<code>highlight_ticks(color)</code>	Change color of Ticks
<code>reset_style()</code>	Set everything's color to default

```
code > my_extension.py
10 import matplotlib.pyplot as plt #import to create figure and axes and to display them
11 import matplotlib.axis as maxis #imported to extend it
12
13
14
15 class MyCustomAxis(maxis.XAxis): #inherits from a class of axis
16     """
17     what it do:-
18     It adds four new methods:
19     set_custom_label() : set a custom colored label on the axis
20     highlight_ticks() : change the color of all tick marks
21     reset_style() : reset label and ticks back to default
22     get_tick_count() : return how many ticks are on the axis
23
24     """
25
26     def __init__(self, axes, *args, **kwargs):
27         super().__init__(axes, *args, **kwargs) #to access feature of parent
28         self.custom_label_text = "" # stores our custom label
29         self.highlight_color = "black" # stores current tick color
30         self._original_color = "black" # stores original color for reset
31
32
33     def set_custom_label(self, text, color="blue"): #for custom label color
34         self.custom_label_text = text
35         self.set_label_text(text) #to set text of label
36         self.label.set_color(color) #to set color of label
37
38     def highlight_ticks(self, color="red"): #this method will highlight the ticks in given color
39         self._highlight_color = color
40         for tick in self.get_major_ticks():
```

Then after that we also test this new class by writing a demo code which use this new class and its methods.

A screenshot of a code editor window titled 'OOP_FinalProject_of_matplotlib'. The editor shows a Python file named 'Demo.py' with the following code:

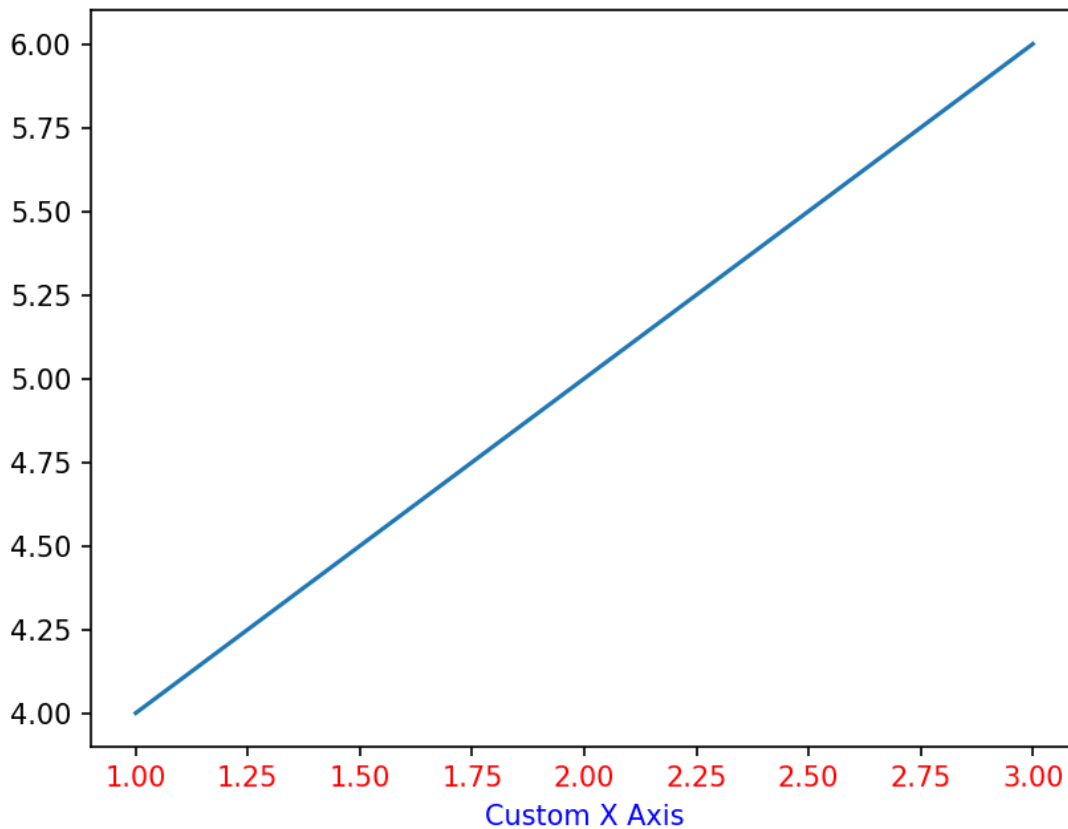
```
code > Demo.py > ...
1  import matplotlib.pyplot as plt
2
3  # import custom class
4  from my_extension import MyCustomAxis
5
6
7
8
9  # Create figure and axes
10 fig, ax = plt.subplots()
11
12 # Use existing xaxis from matplotlib
13 custom_x = ax.xaxis
14
15 # Change class to our custom class
16 custom_x.__class__ = MyCustomAxis
17
18
19 # Plot simple data
20 ax.plot([1, 2, 3], [4, 5, 6])
21
22
23 # Use custom methods
24 custom_x.set_custom_label("Custom X Axis", color="blue")
25
26 custom_x.highlight_ticks("red")
27
28
29 # Show graph
30 plt.show()
```

The editor interface includes a menu bar (File, Edit, Selection, View, Go, Run, ...), a toolbar with icons for file operations, and a sidebar with icons for search, source control, and other tools. The status bar at the bottom shows 'main*' and other editor metrics.

In this section, we extended matplotlib's **Axis** class by creating a new class called **MyCustomAxis**. This class inherits from Axis which means it gets all the original Axis features and we just added new ones on top.

We added **3 new methods** in this class. First method **set_custom_label()** allows us to set a label on the axis with any color we want, which the original Axis class does not support directly. Second method **highlight_ticks()** changes the color of all tick marks and tick numbers on the axis at once. Third method **reset_style()** resets everything back to the original black color with just one method call.

Figure 1



And finally we successfully Extend the code.

Section 7 — References

https://matplotlib.org/stable/api/axis_api.html
https://matplotlib.org/stable/api/lines_api.html
https://matplotlib.org/stable/api/patches_api.html
https://matplotlib.org/stable/api/figure_api.html
https://matplotlib.org/stable/api/artist_api.html
<https://youtu.be/UO98IJQ3QGI?si=f7ORoj6kPYcoMFgb>